

Input/Output

Antonio Pecchia

antonio.pecchia@unisannio.it

Dipartimento di Ingegneria
Laurea in Ingegneria Informatica
Sistemi Operativi



UNIVERSITÀ DEGLI STUDI
DEL SANNIO Benevento

CONCETTI INTRODUTTIVI

Cap 5 fino a par. 5.1.3 (incluso)

Dispositivi di Input/Output (I/O)

- Oltre a supportare l'astrazione di *processi*, *spazio degli indirizzi e file*, il sistema operativo controlla i **dispositivi di I/O**.

Dispositivo	Velocità di trasferimento
Tastiera	10 byte/s
Mouse	100 byte/s
Modem a 56 K	7 KB/s
Bluetooth 5 BLE	256 KB/s
Scanner a 300 dpi	1 MB/s
Videocamera digitale	3,5 MB/s
Wireless 802.11n	37,5 MB/s
USB 2.0	60 MB/s
Disco Blu-ray 16x	72 MB/s
Gigabit Ethernet	125 MB/s
Disco fisso SATA 3	600 MB/s
USB 3.0	625 MB/s
Bus PCIe 3.0 single lane	985 MB/s
Wireless 802.11ax	1,25 GB/s
SSD NVME PCIe Gen 3.0 (lettura)	3,5 GB/s
USB 4.0	5 GB/s
PCI Express 6.0	126 GB/s

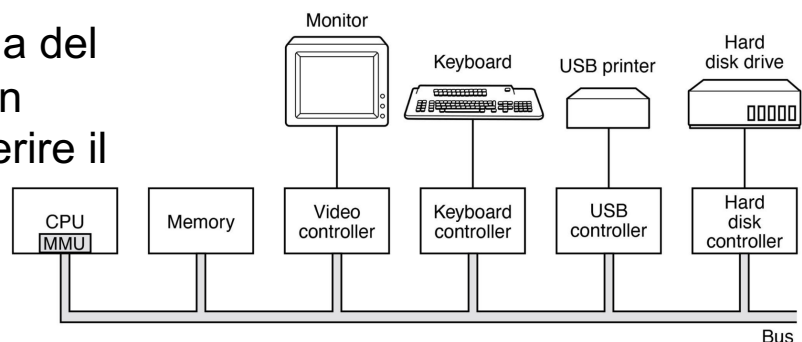
Figura 5.1 Velocità di trasferimento dei dati di alcuni dispositivi tipici, reti e bus.

Dispositivi di Input/Output (I/O)

- ❑ Categorie di dispositivi di I/O:
 - **dispositivi a blocchi**: archivia informazioni in blocchi (ciascuno con il proprio indirizzo), i trasferimenti sono in unità di uno o più blocchi; ciascun blocco può essere letto-scritto indipendentemente dagli altri (per es., HDD, SSD).
 - **dispositivi a caratteri**: rilasciano/accettano un flusso di caratteri, senza alcuna struttura a blocchi; non è indirizzabile (per es., stampante, interfaccia di rete, mouse).
 - **altro**: i clock.

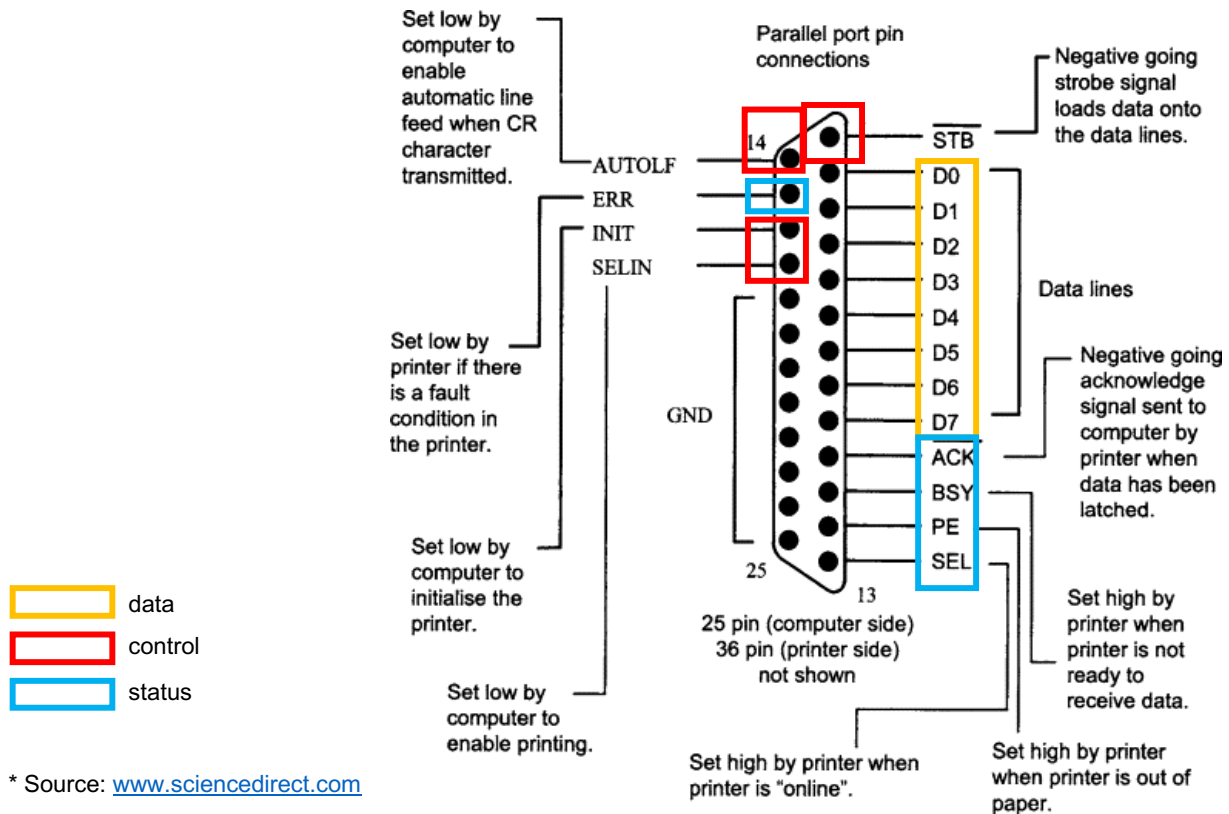
Controller del dispositivo

- ❑ **Controller** o *adattatore (adapter)*: parte elettronica tra il computer ed il dispositivo di I/O "vero e proprio";
- ❑ Controlla il dispositivo ed accetta comandi dal S.O.;
- ❑ Chip sulla scheda madre o scheda inseribile in uno slot di espansione:
 - in generale, la scheda del controller presenta un **connettore** in cui inserire il cavo del dispositivo.

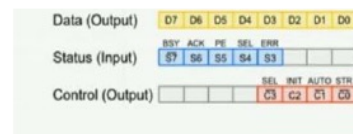
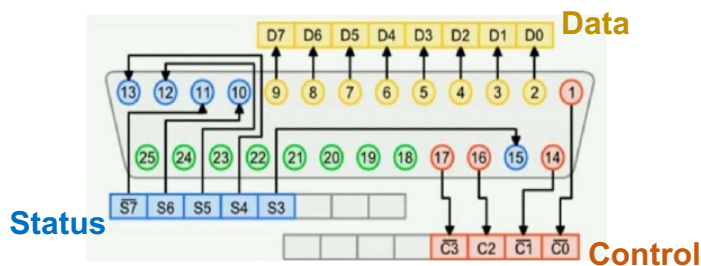
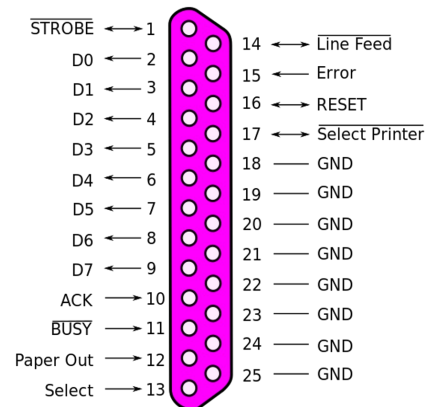


Controller del dispositivo

Parallel Port

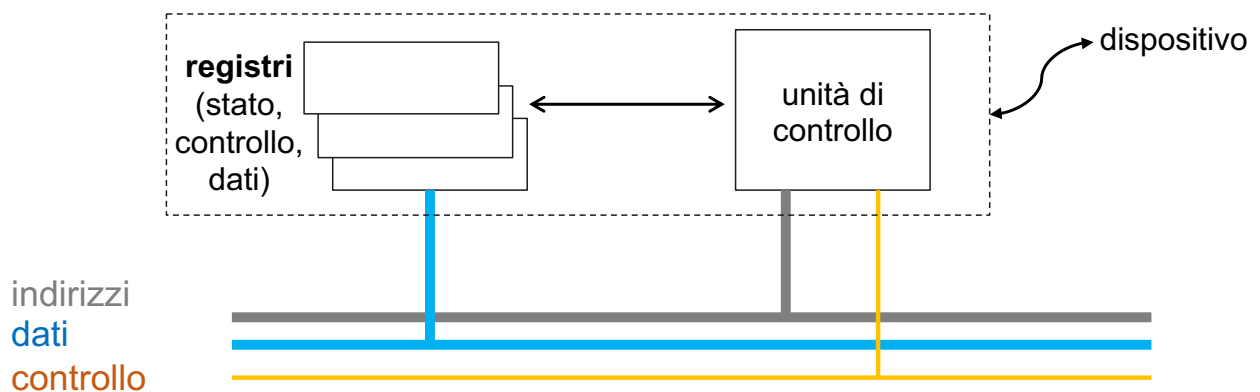


Controller del dispositivo



Controller del dispositivo

- Ogni controller dispone di **alcuni registri** usati per le comunicazioni con la CPU. Schema esemplificativo:



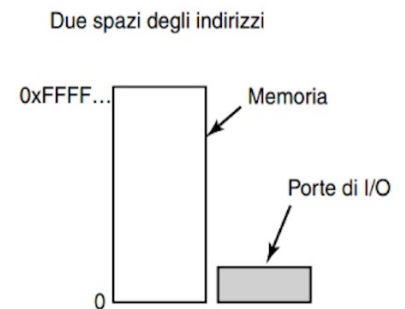
Comunicazione CPU-registri

- Primo approccio: **port-mapped I/O** (o "isolated" I/O);
- I registri di I/O sono acceduti con **numeri di porta** dedicati ed **istruzioni specifiche** per l'I/O:

```
IN REG, PORT //leggi PORT e salva in REG
OUT PORT, REG //scrivi REG in PORT
```

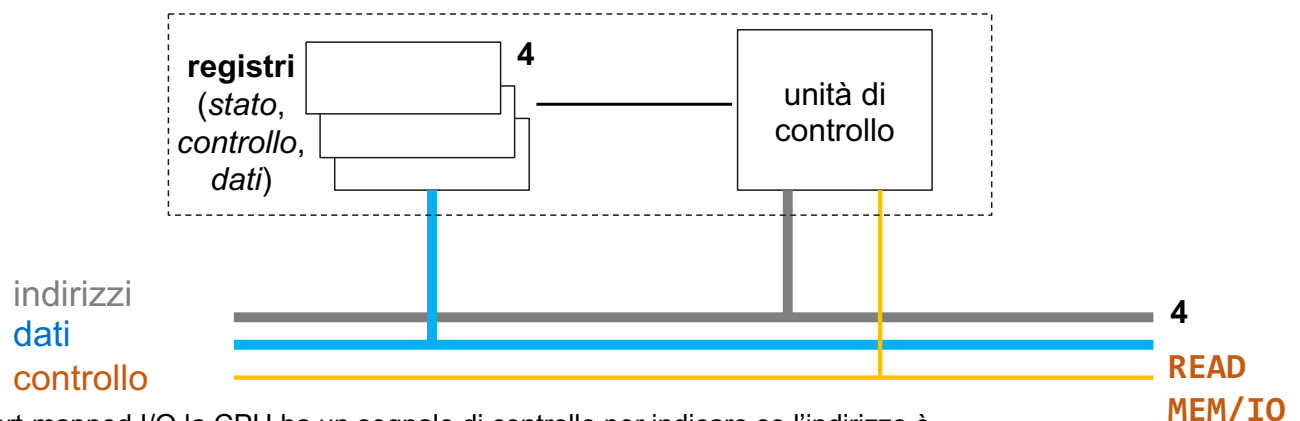
Per es., è possibile avere:

```
IN R0, 4
MOV R0, 4
```



Controller del dispositivo

- Ogni controller dispone di **alcuni registri** usati per le comunicazioni con la CPU. Schema esemplificativo:



* In port-mapped I/O la CPU ha un segnale di controllo per indicare se l'indirizzo è una locazione di memoria o un dispositivo di I/O.

Comunicazione CPU-registri

- ❑ Secondo approccio: **memory-mapped I/O**;
- ❑ A ciascun registro di I/O è assegnato un **indirizzo di memoria** univoco a cui non è "assegnata" memoria;
- ❑ I registri di I/O sono acceduti come fossero memoria.

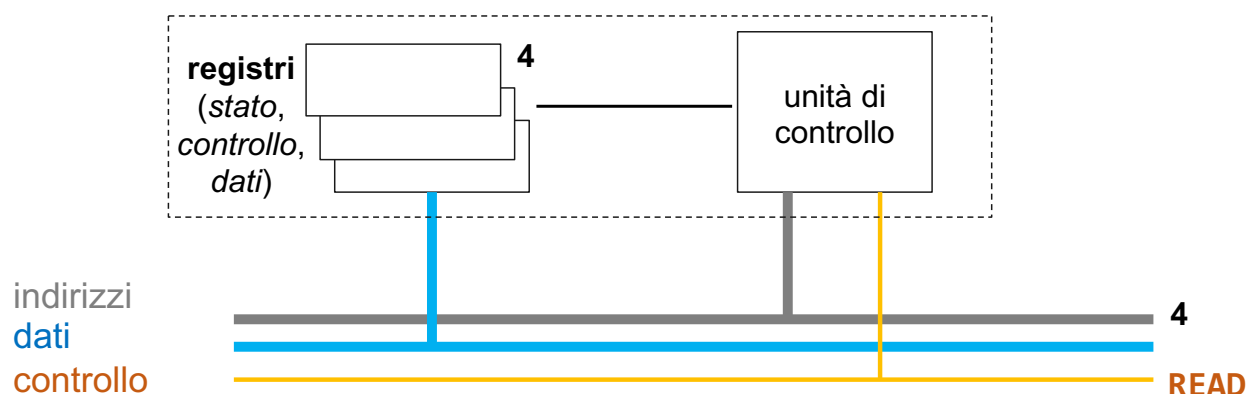
```
mov 0x278, ax    # store value of ax  
                # to address 0x278
```

Spazio degli indirizzi singolo



Controller del dispositivo

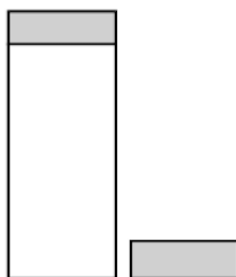
- ❑ Ogni controller dispone di **alcuni registri** usati per le comunicazioni con la CPU. Schema esemplificativo:



Comunicazione CPU-registri

- Terzo approccio: **schema ibrido** (coesistenza di entrambe le soluzioni nella stessa architettura).

Due spazi degli indirizzi



Vantaggi e svantaggi

- Vantaggi** memory-mapped vs port-mapped: no istruzioni I/O specifiche, **protezione semplificata***, semplicità di programmazione;
- Svantaggi** memory-mapped vs port-mapped: caching delle parole della memoria, *esaminare* i riferimenti alla memoria.

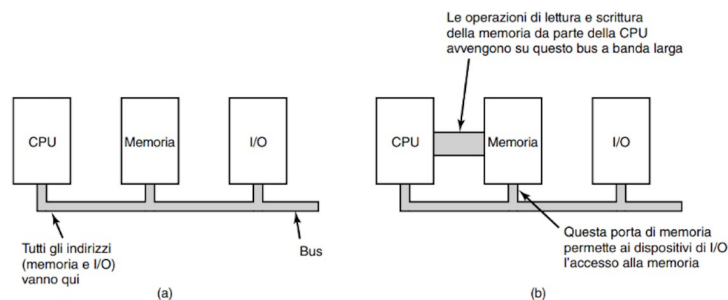
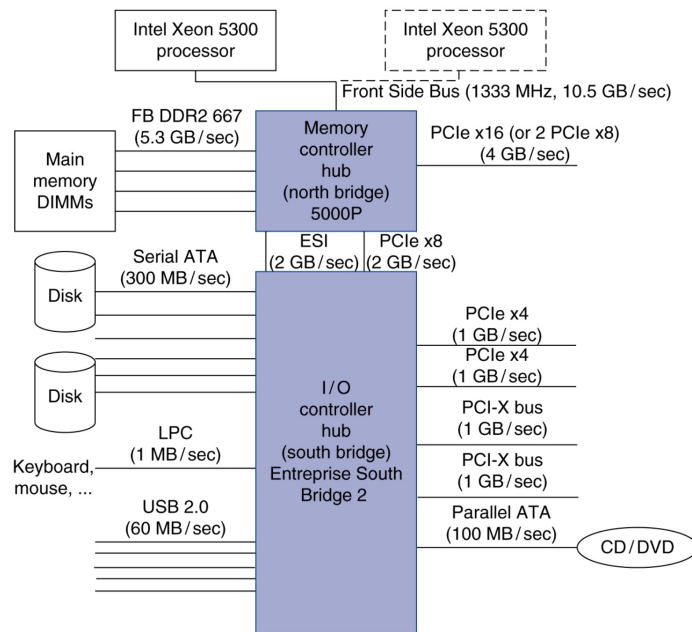
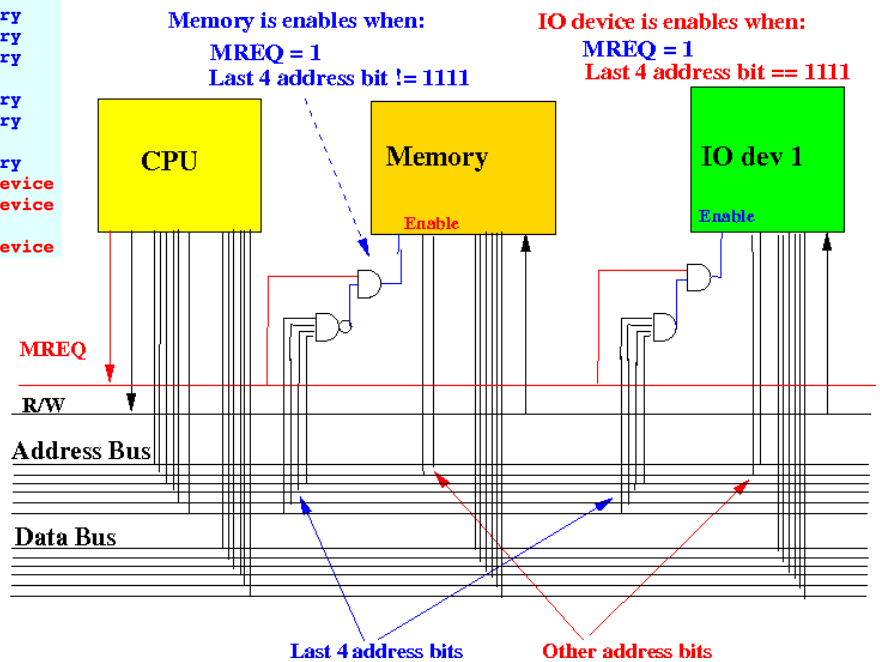


Figura 5.3 (a) Architettura a bus singolo.

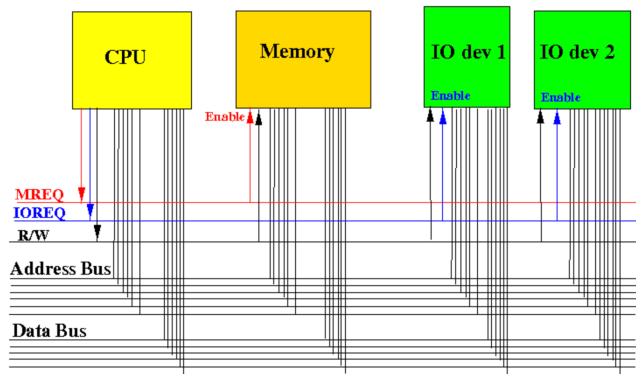
*E' sufficiente che il sistema operativo "eviti" di mettere la parte dello spazio degli indirizzi contenenti i registri di I/O nello spazio degli indirizzi virtuali di un qualunque utente.



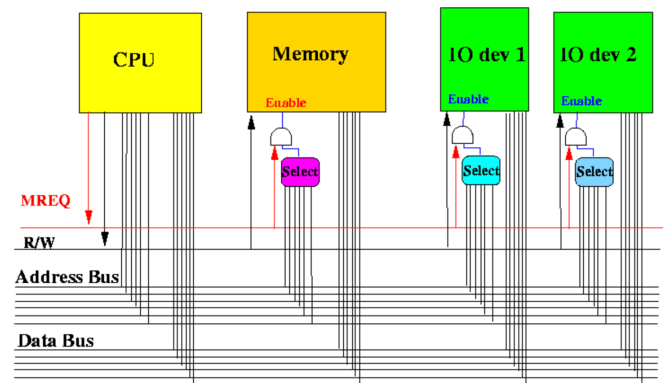
Address (binary)	Use for
00000000 00000000 00000000 00000000	Memory
00000000 00000000 00000000 00000001	Memory
00000000 00000000 00000000 00000011	Memory
....	
11100000 00000000 00000000 00000000	Memory
11100000 00000000 00000000 00000000	Memory
....	
11101111 11111111 11111111 11111111	Memory
11110000 00000000 00000000 00000000	IO device
11110000 00000000 00000000 00000001	IO device
....	
11111111 11111111 11111111 11111111	IO device



* <https://www.cs.emory.edu/~cheung/Courses/355/Syllabus/6-io/addressing.html>



Port-mapped: la CPU ha un segnale esplicito per specificare che l'indirizzo sia una locazione di memoria, o un dispositivo di I/O.



Memory-mapped: *non* c'è un segnale esplicito per indicare che un indirizzo sia memoria o I/O; la selezione di base sul valore dell'indirizzo stesso (NOTA: i set di indirizzi sono disgiunti).

* <https://www.cs.emory.edu/~cheung/Courses/355/Syllabus/6-io/addressing.html>

PROGRAMMAZIONE DELL'I/O

Cap 5 par 5.1.4 (Direct Memory Access)

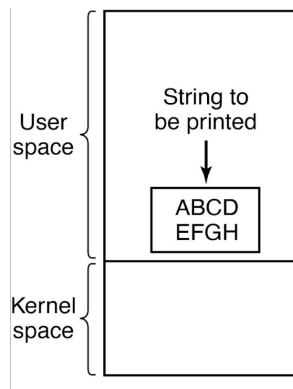
Cap 5 par 5.2.2, 5.2.3, 5.2.4

Eseguire operazione di I/O

- ▣ Per eseguire l'I/O esistono tre metodi principali:
 - I/O programmato;
 - I/O guidato dagli interrupt;
 - I/O con DMA (Direct Memory Access).

I/O programmato

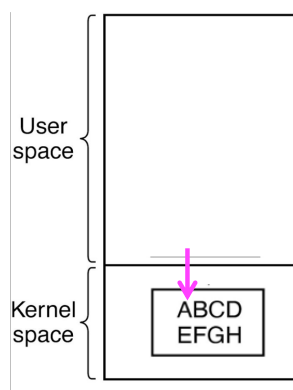
- Il processo attende (**busy-wait**) il completamento dell'operazione di I/O. Esempio:



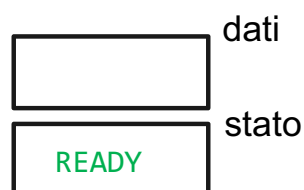
```
if ( printer_open(...) ) {           // system call
    printer_print(buffer, ... )      // system call
    ...
```

I/O programmato

- Il processo attende (**busy-wait**) il completamento dell'operazione di I/O. Esempio:

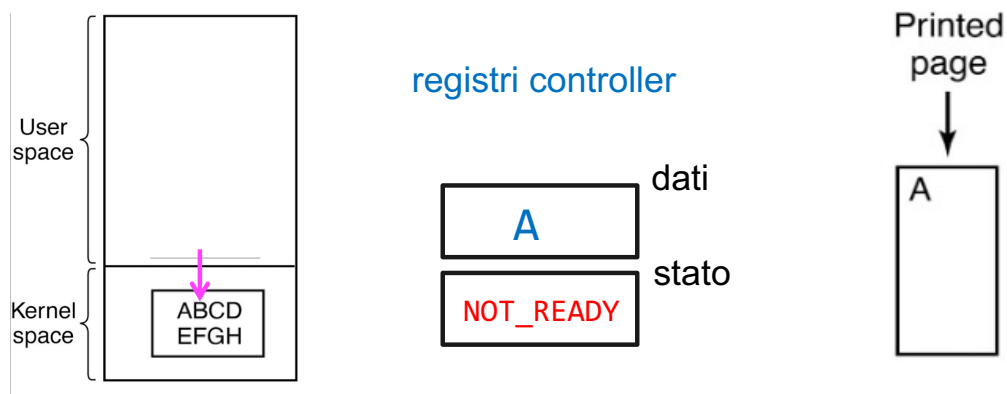


registri controller



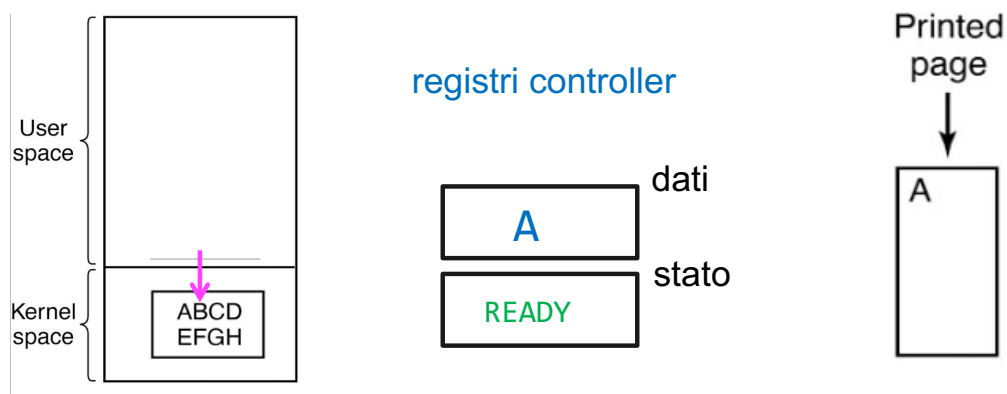
I/O programmato

- Il processo attende (**busy-wait**) il completamento dell'operazione di I/O. Esempio:



I/O programmato

- Il processo attende (**busy-wait**) il completamento dell'operazione di I/O. Esempio:



I/O programmato

- Il processo attende (**busy-wait**) il completamento dell'operazione di I/O. Esempio:

```
copy_from_user(buffer, p, count);
for (i = 0; i < count; i++) {
    while (*printer_status_reg != READY) ;
    *printer_data_register = p[i];
}
return_to_user();
```

/* p is the kernel buffer */
/* loop on every character */
/* loop until ready */
/* output one character */

*Nell'esempio si assume il caso di I/O mappato in memoria

I/O guidato dagli interrupt

- Viene invocato un comando di I/O, il processo che ha invocato il comando viene sospeso;
- Il modulo di I/O solleva un **interrupt** quando ha terminato.

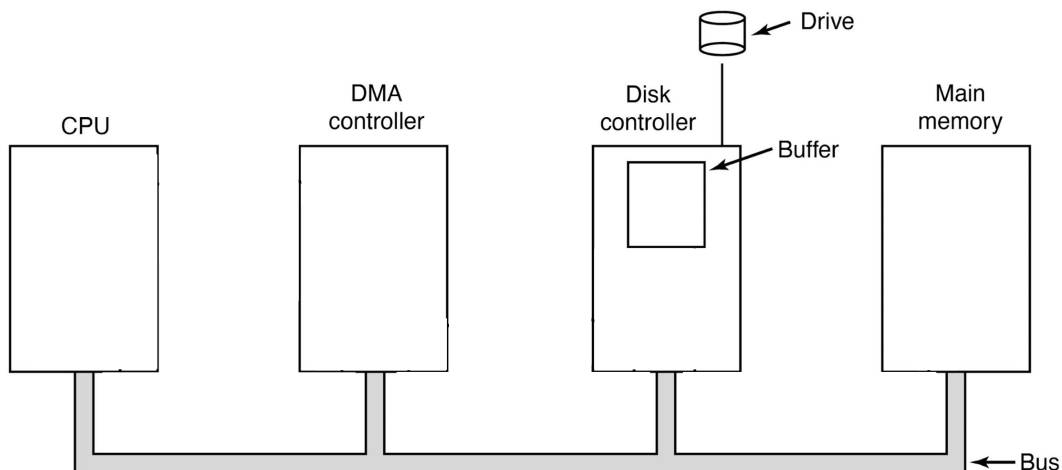
```
copy_from_user(buffer, p, count);
enable_interrupts();
while (*printer_status_reg != READY) ;
*printer_data_register = p[0];
scheduler();
```

procedura di
servizio interrupt →

```
if (count == 0) {
    unblock_user();
} else {
    *printer_data_register = p[i];
    count = count - 1;
    i = i + 1;
}
acknowledge_interrupt();
return_from_interrupt();
```

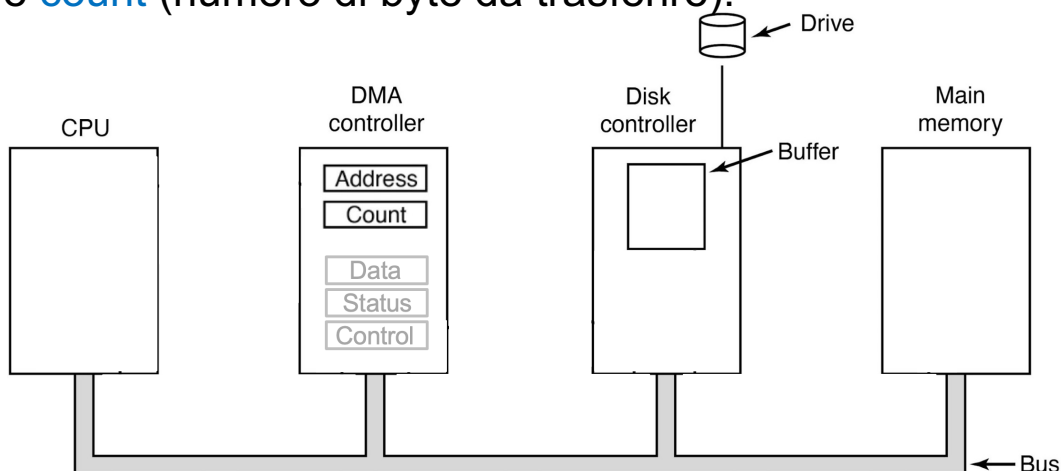
Direct Memory Access (DMA)

- ❑ Il DMA ha accesso al bus *indipendentemente* dalla CPU;
- ❑ Possiede registri che possono essere letti-scritti dalla CPU.



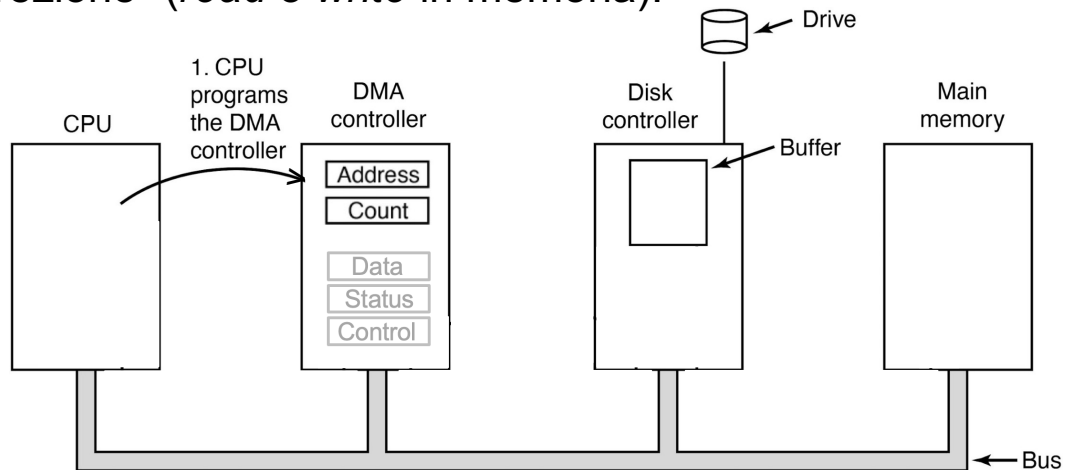
Direct Memory Access (DMA): registri

- ❑ I "consueti" data / status / control più, in aggiunta, **address** (**indirizzo iniziale** di memoria *in cui* – oppure *da cui* – trasferire i dati) e **count** (numero di byte da trasferire).



Direct Memory Access (DMA): funzionamento

1. La CPU "**programma**" il controller DMA, impostando i registri per definire dove (o *da dove*) trasferire, quanto, il dispositivo coinvolto, la "direzione" (*read o write in memoria*).

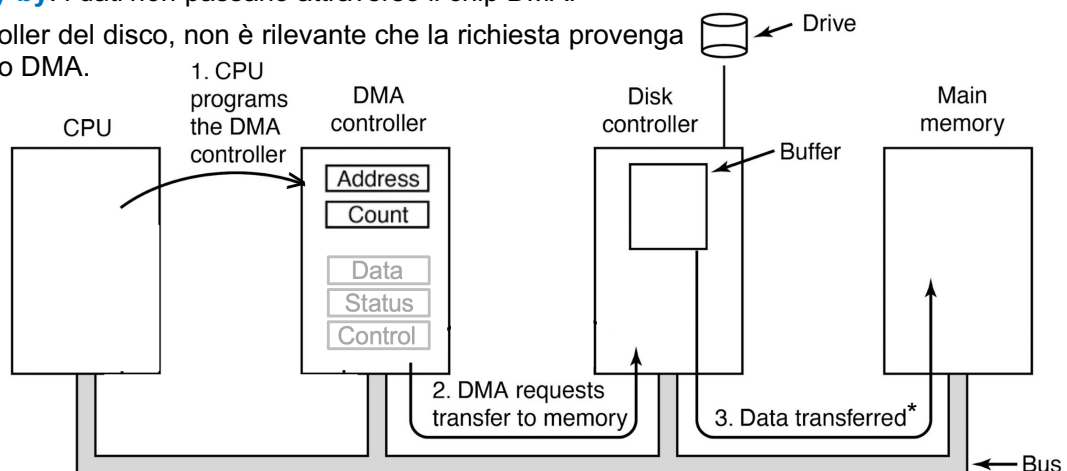


Direct Memory Access (DMA): funzionamento

2. Il DMA invia una richiesta di lettura** al controller disco;
3. Scrittura in memoria (l'indirizzo in cui scrivere è sulle linee di bus);

* modalità **fly-by**: i dati *non* passano attraverso il chip DMA.

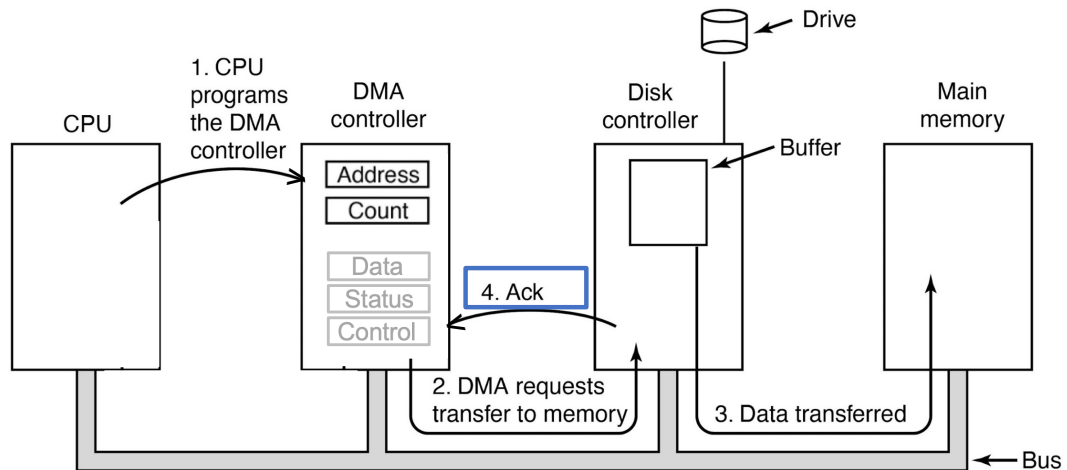
** per il controller del disco, non è rilevante che la richiesta provenga dalla CPU o DMA.



Direct Memory Access (DMA): funzionamento

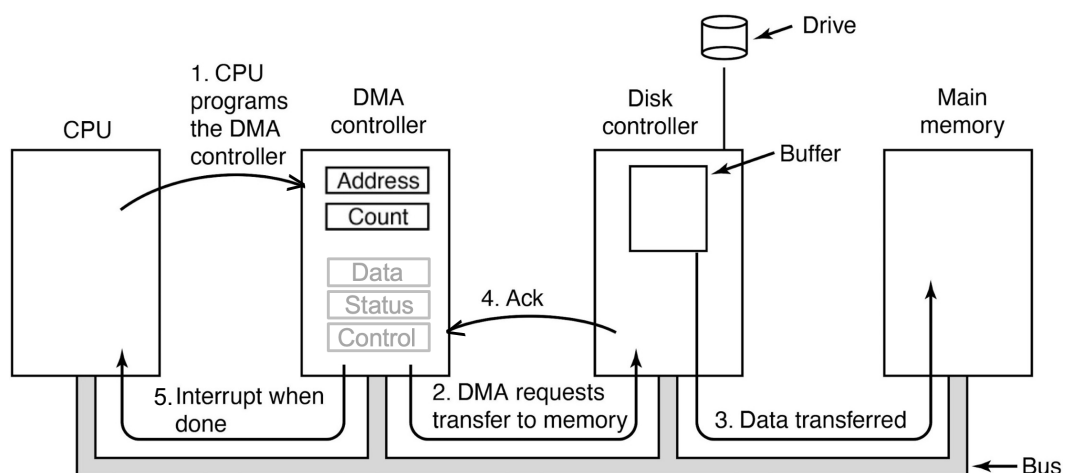
4. Il controller disco manda un segnale di conferma ('Ack') al DMA (decremento contatore byte);

→ ripetizione step 2-4 finché ci sono dati da trasferire (count > 0)



Direct Memory Access (DMA): funzionamento

5. Il DMA invia un interrupt alla CPU.



I/O con DMA

- Un dispositivo **Direct Memory Access (DMA)** può controllare lo scambio di dati tra la memoria principale ed il dispositivo di I/O;
- Il processore viene interrotto solo quando l'intero blocco di dati è stato trasferito.

I/O con DMA

- Nell'esempio della stampante **con DMA**, si lascia che sia il DMA ad inviare i caratteri alla stampante.

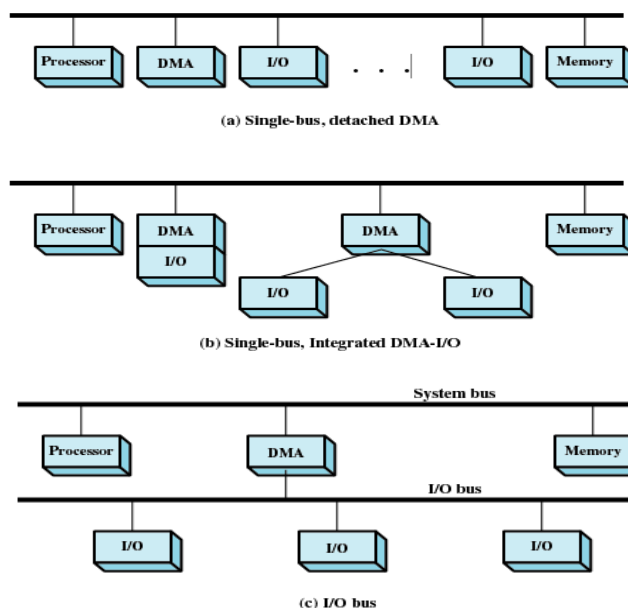
```
copy_from_user(buffer, p, count);  
set_up_DMA_controller( );  
scheduler( );
```

```
acknowledge_interrupt( );  
unblock_user( );  
return_from_interrupt( );
```

procedura di
servizio interrupt

*(i dati richiesti sono stati già
trasferiti)*

Configurazioni DMA



Caratteristiche dei DMA

- ❑ I DMA possono operare più trasferimenti insieme;
- ❑ **Word-at-a-time** (cycle stealing) o a **blocco** (burst);
- ❑ Modalità di funzionamento:
 - **fly-by**: i dati *non* passano attraverso il chip DMA – possibile per trasferimenti che coinvolgono I/O e memoria;
 - **flow-through**: i dati passano prima attraverso il chip DMA, il DMA effettua una seconda richiesta al bus per inviare i dati a destinazione – supporta anche trasferimenti/copia tra I/O-I/O e memoria-memoria.
- ❑ La maggior parte dei controller DMA usa, per i trasferimenti, indirizzi di memoria **fisici***.

*Il sistema operativo converte l'indirizzo virtuale del buffer di memoria necessario in un indirizzo fisico, e scrive, tale indirizzo fisico, nel controller DMA.

LIVELLI DEL SOFTWARE DI I/O

Cap 5 par 5.3 (esclusi 5.3.1 e 5.3.4)

Livelli del software di I/O

- Il software di I/O è tipicamente organizzato in livelli

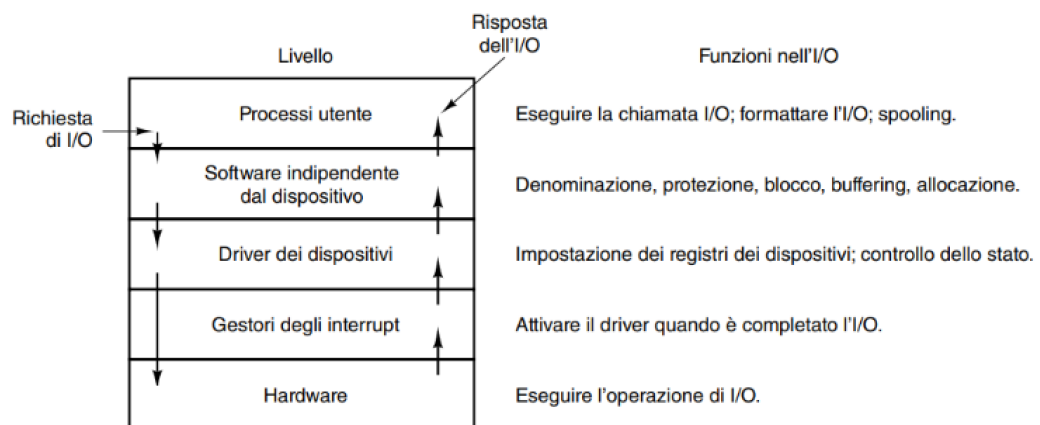


Figura 5.17 Livelli del sistema di I/O e principali funzioni di ogni livello.

Driver dei dispositivi (device driver)

- Codice (tipicamente scritto dal produttore del device) per il controllo di ciascun dispositivo di I/O collegato ad un computer.

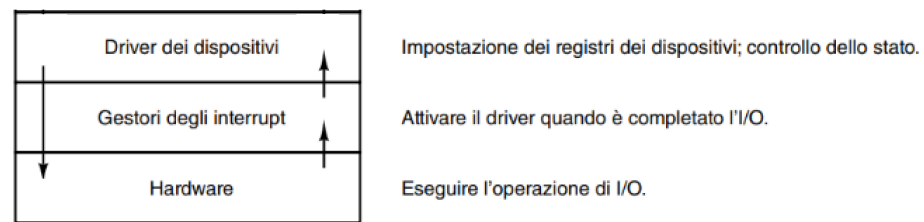


Figura 5.17 Livelli del sistema di I/O e principali funzioni di ogni livello.

Driver dei dispositivi (device driver)

- Alcune **funzioni** di un driver (elenco non esaustivo):
 - inizializzazione del dispositivo;
 - accetta richieste di lettura-scrittura da un *chiamante*;
 - verifica dei parametri di input;
 - invio comandi al device (scrittura nei registri del controller);
- **scenari possibili:**
 - necessità di attendere il completamento del dispositivo: il driver si blocca finché arriva un interrupt;
 - l'operazione termina senza ritardi.
- controllo di eventuali errori e restituzione dati al *chiamante*.

Driver dei dispositivi (device driver)

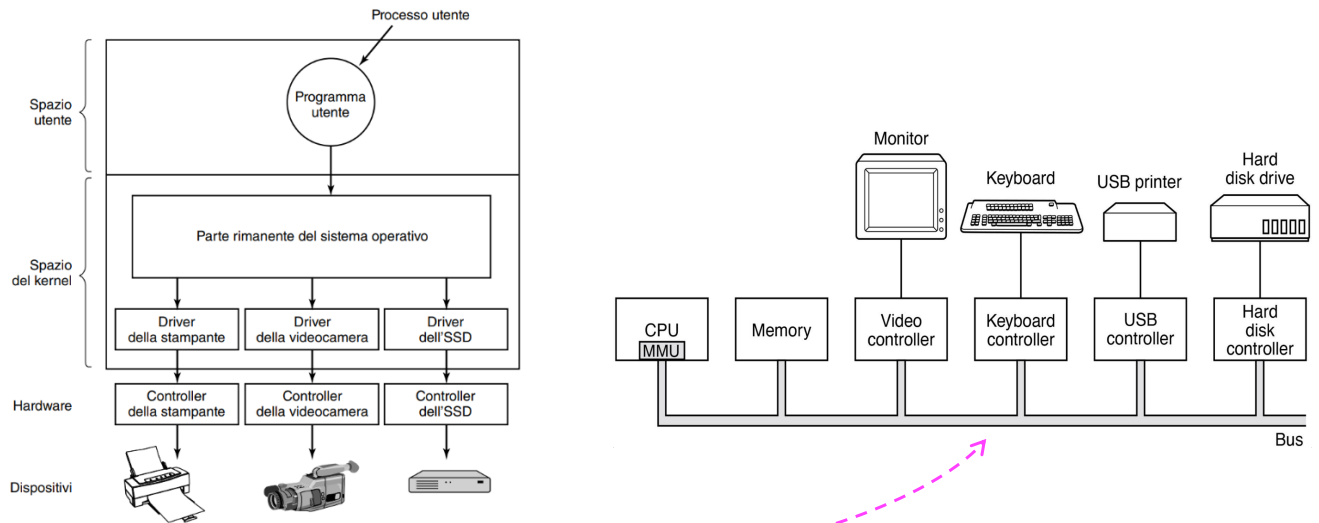


Figura 5.12 Posizionamento logico dei driver dei dispositivi. In realtà tutte le comunicazioni tra i driver e i controller dei dispositivi avvengono sul bus.

Altri aspetti

- ❑ In generale, il driver è **parte del kernel**:
 - il driver può essere eseguito solamente effettuando una system call;
 - fanno "eccezione" alcuni kernel (per es. MINIX, a microkernel) dove i driver sono eseguiti come procedure utente (con chiamate di sistema per la lettura/scrittura dei registri dei dispositivi).
- ❑ I sistemi operativi supportano il caricamento dei driver durante l'esecuzione.

Software indipendente dal dispositivo

- Implementa funzioni di I/O **comuni** a tutti i dispositivi ed offre una interfaccia uniforme al livello utente.

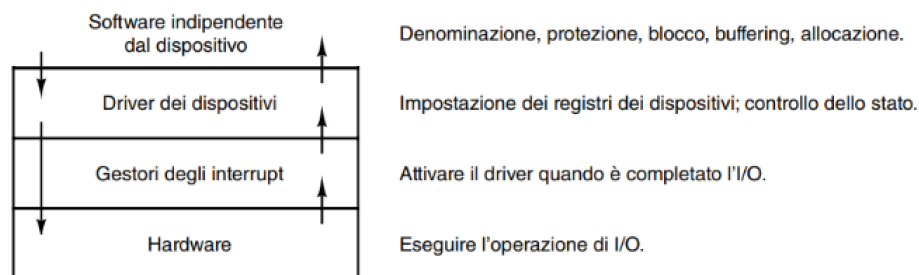


Figura 5.17 Livelli del sistema di I/O e principali funzioni di ogni livello.

Livello utente

- Tipicamente, **librerie** collegate con i programmi utente; chiamate di sistema eseguite da procedure di libreria.

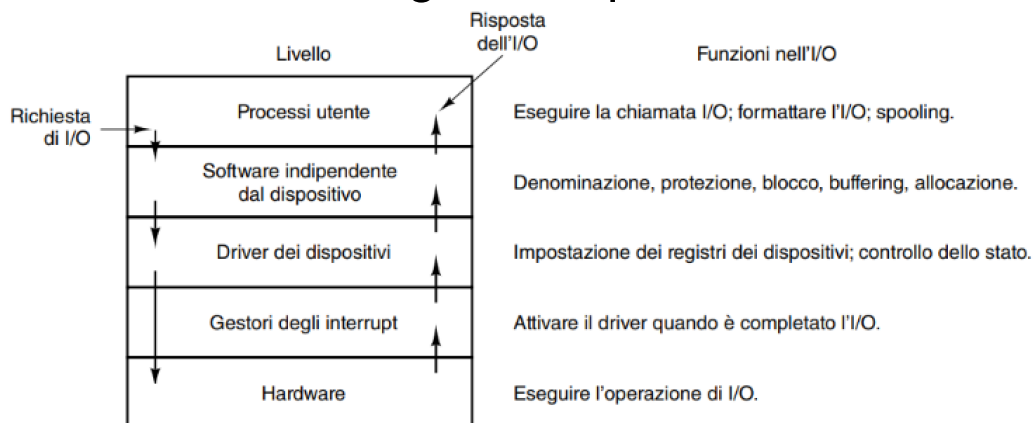
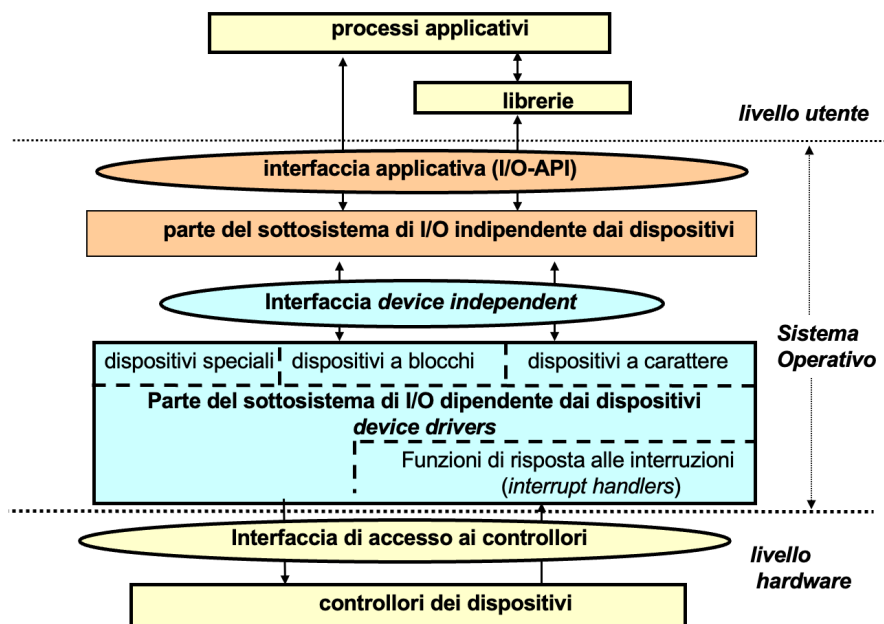


Figura 5.17 Livelli del sistema di I/O e principali funzioni di ogni livello.

Una ulteriore rappresentazione



Software indipendente dal dispositivo

- ▣ Funzioni del software per l'I/O indipendente dal dispositivo.

Interfacciamento uniforme dei driver dei dispositivi
Buffering
Segnalazione degli errori
Allocazione e rilascio dei dispositivi dedicati
Dimensione dei blocchi indipendente dal dispositivo

Figura 5.13 Funzioni del software per l'I/O indipendente dai dispositivi.

Driver e software indipendente dal dispositivo

- ❑ I driver operano in accordo ad un *modello* definito di come essi interagiscono con il SO;
- ❑ I SO assumono i "classificano" i driver in un numero *limitato* di categorie:
 - dispositivi a **blocchi** e dispositivi a **caratteri**.
- ❑ I SO definiscono una **interfaccia standard** che deve essere supportata dai dispositivi a blocchi e quelli a caratteri (per es., "leggi blocco", "scrivi stringa caratteri").

Esempio: https://linux-kernel-labs.github.io/refs/heads/master/labs/block_device_drivers.html#struct-block-device-operations-structure

```
struct block_device_operations {
    int (*open) (struct block_device *, fmode_t);
    int (*release) (struct gendisk *, fmode_t);
    int (*locked_ioctl) (struct block_device *, fmode_t, unsigned,
                        unsigned long);
    int (*ioctl) (struct block_device *, fmode_t, unsigned, unsigned long);
    int (*compat_ioctl) (struct block_device *, fmode_t, unsigned,
                        unsigned long);
    int (*direct_access) (struct block_device *, sector_t,
                        void **, unsigned long *);
    int (*media_changed) (struct gendisk *);
    int (*revalidate_disk) (struct gendisk *);
    int (*getgeo) (struct block_device *, struct hd_geometry *);
    blk_qc_t (*submit_bio) (struct bio *bio);
    struct module *owner;
}
```

Driver e software indipendente dal dispositivo

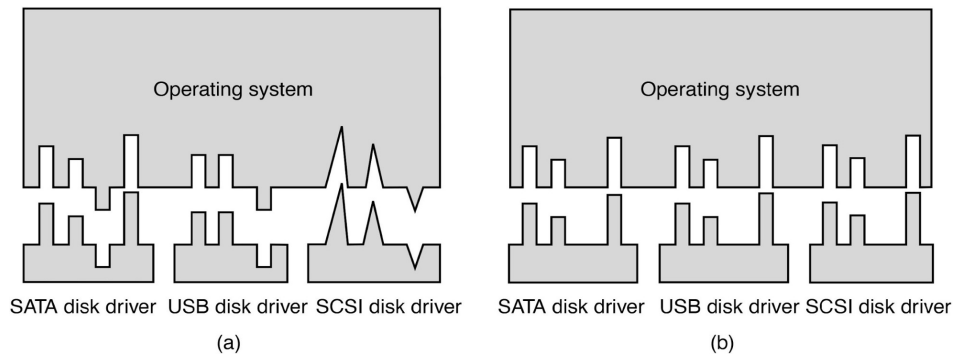
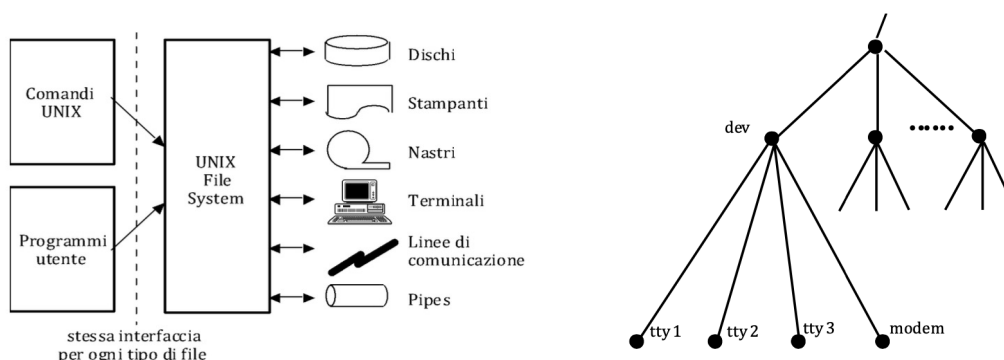


Figure 5-14. (a) Without a standard driver interface. (b) With a standard driver interface.

Denominazione (naming) dei dispositivi di I/O

- ❑ In UNIX, ogni device di I/O viene visto, a tutti gli effetti, come un **file (file speciale)**;
- ❑ Richieste di lettura/scrittura da/a file speciali causano operazioni di input/output dai/ai devices associati.



Denominazione (naming) dei dispositivi di I/O

- I file speciali si trovano nella directory **dev**;
- Sono caratterizzati da *tipo* (a blocchi/caratteri), *classe dispositivo* (**major device number**) e *istanza dispositivo* (**minor device number**):

```
ls -la /dev/sda*
```

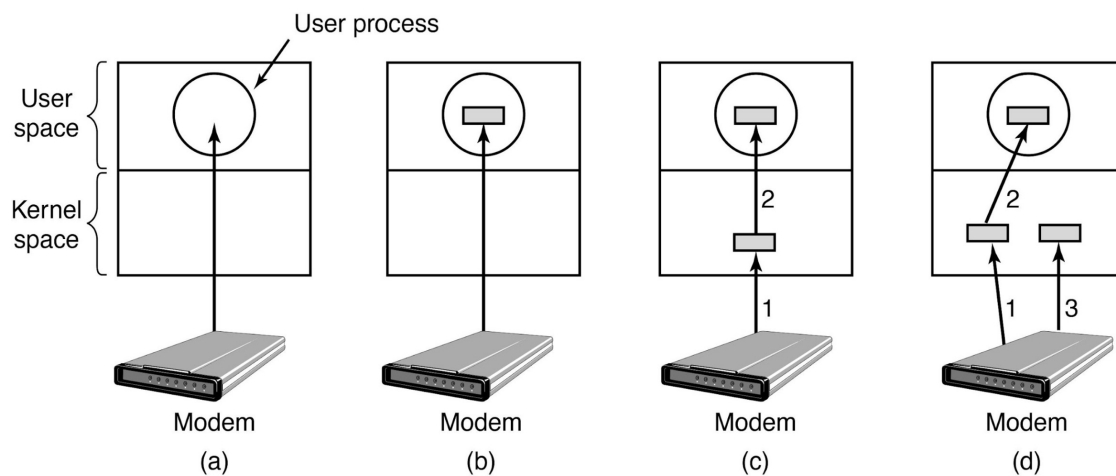
- L'apertura di un file speciale restituisce un **descrittore**, che punta a una **struct** per effettuare lettura-scrittura blocchi o flussi di caratteri dal/sul dispositivo:
 - l'interfaccia di accesso è **uniforme**; il driver implementa le funzionalità.

Denominazione (naming) dei dispositivi di I/O

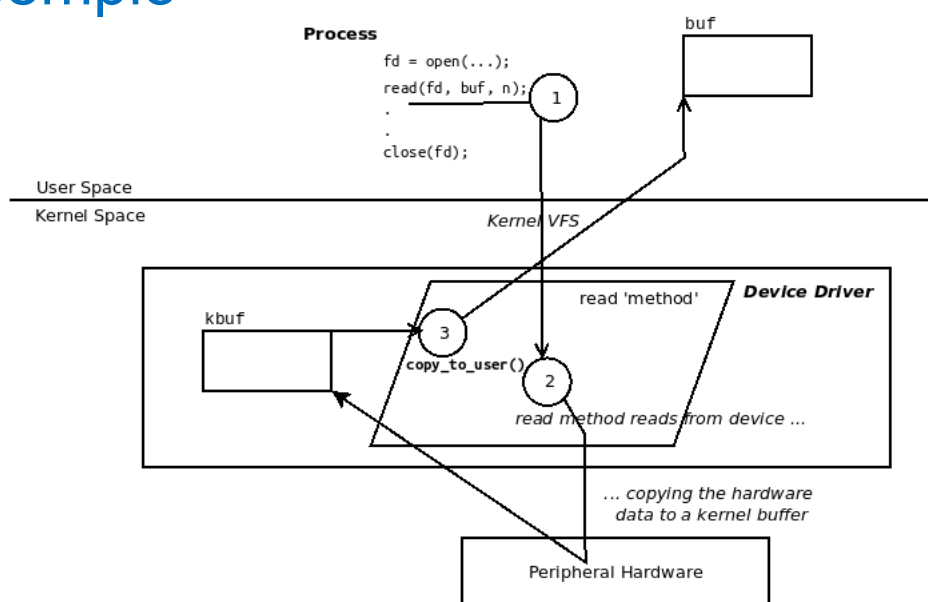
Table 13-7. Examples of device files

Name	Type	Major	Minor	Description
/dev/fd0	block	2	0	Floppy disk
/dev/hda	block	3	0	First IDE disk
/dev/hda2	block	3	2	Second primary partition of first IDE disk
/dev/hdb	block	3	64	Second IDE disk
/dev/hdb3	block	3	67	Third primary partition of second IDE disk
/dev/tty0	char	3	0	Terminal
/dev/console	char	5	1	Console
/dev/lp1	char	6	1	Parallel printer
/dev/ttyS0	char	4	64	First serial port
/dev/rtc	char	10	135	Real-time clock
/dev/null	char	1	3	Null device (black hole)

Buffering



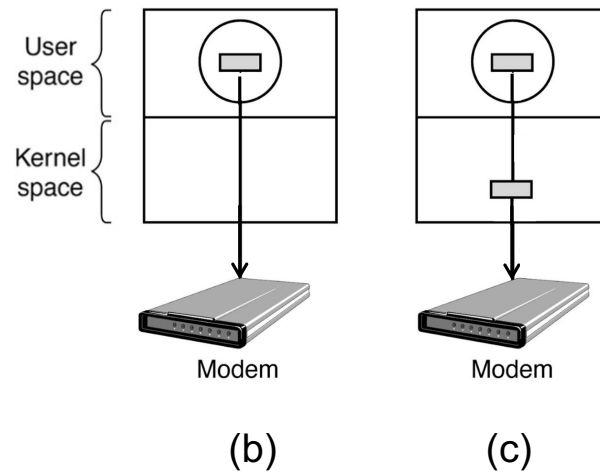
Un esempio*



*Source: [Linux Kernel Programming Part 2](#)

Buffering

- Il buffering è usato anche in **output**; una soluzione tipica è la (c): copia in buffer kernel e sblocco del chiamante.



Buffering

- Il buffering ha pro e *contro*, tra cui il possibile (elevato) numero di copie di un dato.

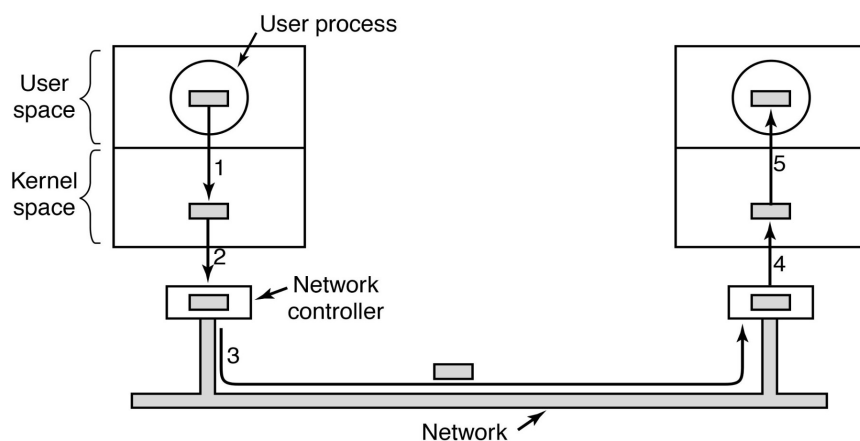


Figure 5-16. Networking may involve many copies of a packet.

Restanti funzioni

- ❑ **Segnalazione errori**: errori di programmazione, errori di I/O;
- ❑ **Allocazione-rilascio** dei dispositivi;
- ❑ **Dimensione dei blocchi**: astrazione della dimensione del blocco "logico", indipendente dalla dimensione del settore fisico.

Interfacciamento uniforme dei driver dei dispositivi
Buffering
Segnalazione degli errori
Allocazione e rilascio dei dispositivi dedicati
Dimensione dei blocchi indipendente dal dispositivo

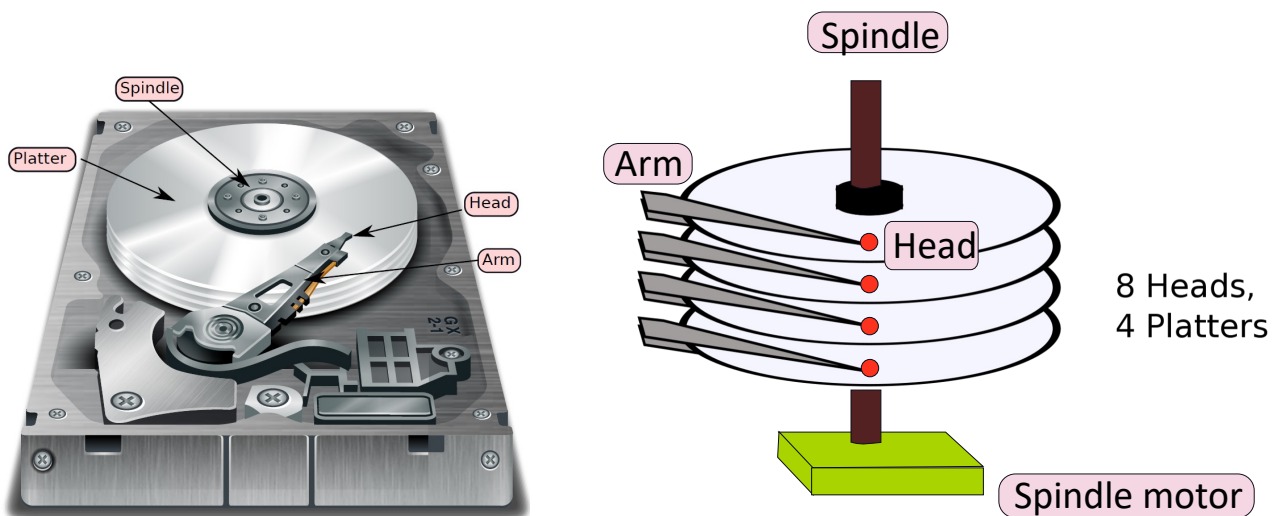
Figura 5.13 Funzioni del software per l'I/O indipendente dai dispositivi.

MEMORIA DI MASSA (DISCHI e SSD)

Cap 5 par 5.4.1
Cap 4 par 4.3.7

(fino ad 'Algoritmo dell'ascensore' incluso)
(escluso 'Garbage collection' in poi)

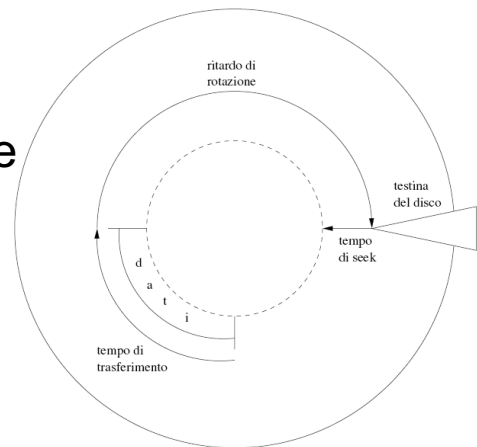
Dischi magnetici*



*I dischi magnetici sono stati già trattati nella lezione sul [file system](#).

Scheduling del braccio del disco (HDD)

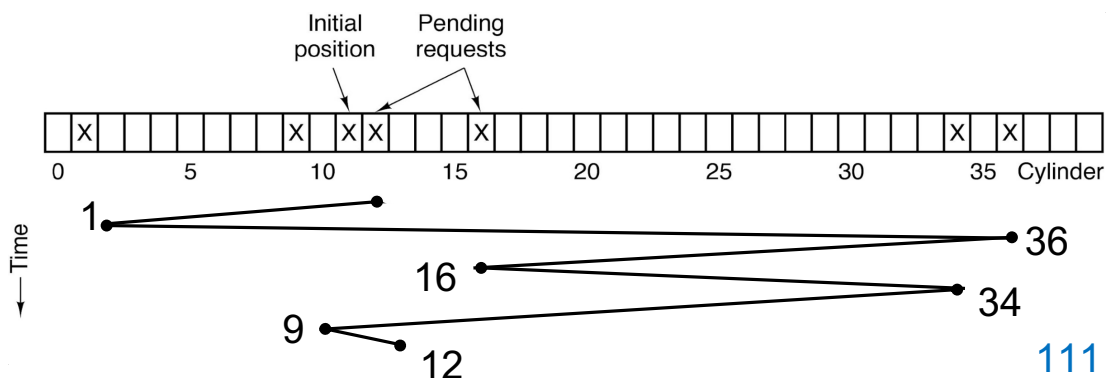
- In generale, il **tempo di ricerca** è decisamente maggiore degli altri.
- Ridurre il tempo di ricerca migliora le prestazioni del sistema.



1. il tempo di ricerca (il tempo per spostare il braccio al cilindro giusto);
2. il ritardo rotazionale (il tempo necessario al settore giusto per ruotare sotto la testina);
3. il tempo del trasferimento vero e proprio dei dati.

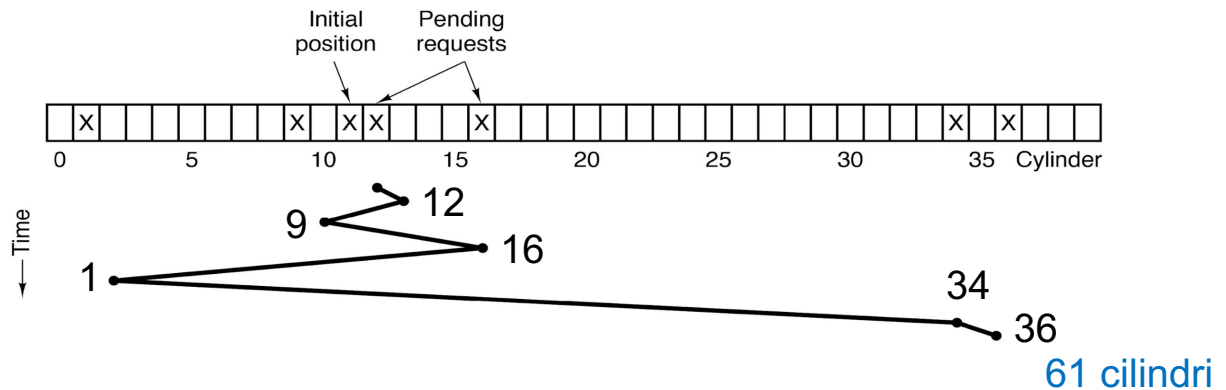
Scheduling del braccio del disco (HDD)

- **FCFS** (First-Come, First Served): le richieste sono servite in ordine, senza tener conto del cilindro:
 - posizione iniziale: 11; richieste = 1, 36, 16, 34, 9, 12.



Scheduling del braccio del disco (HDD)

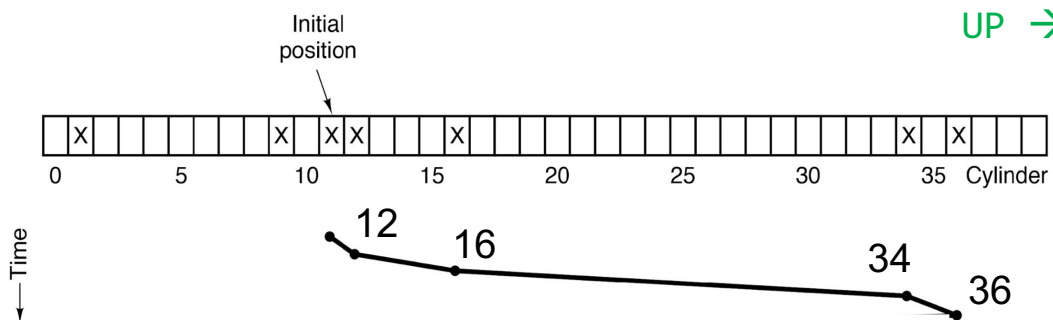
- **SSF** (Shortest Seek First): gestisci, come seguente, la richiesta più vicina:
 - posizione iniziale: 11; richieste = 1, 36, 16, 34, 9, 12.



Scheduling del braccio del disco (HDD)

- **Algoritmo dell'ascensore** (elevator algorithm): si muove in una direzione finché non vi sono più richieste in quella direzione; quindi cambia direzione.
 - posizione iniziale: 11; richieste = 1, 36, 16, 34, 9, 12.

BIT UP/DOWN
UP →

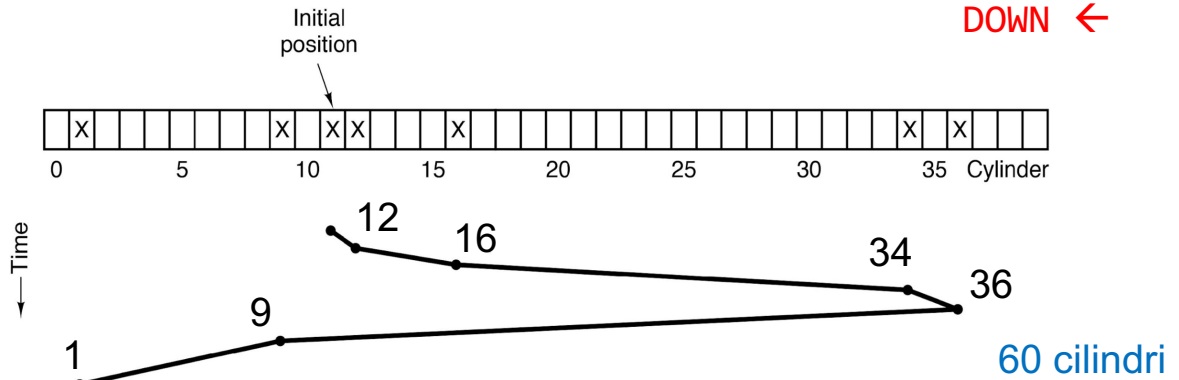


Scheduling del braccio del disco (HDD)

- ❑ **Algoritmo dell'ascensore** (elevator algorithm): si muove in una direzione finché non vi sono più richieste in quella direzione; quindi cambia direzione.

- posizione iniziale: 11; richieste = 1, 36, 16, 34, 9, 12.

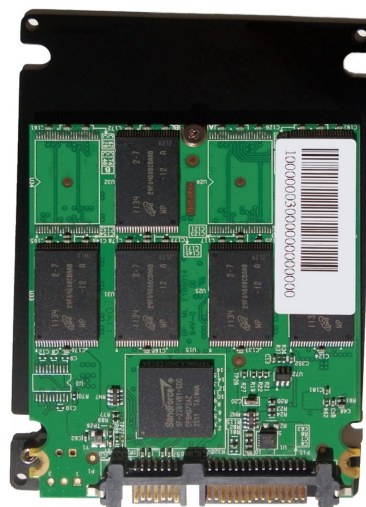
BIT UP/DOWN
DOWN ←



Dischi magnetici vs SSD

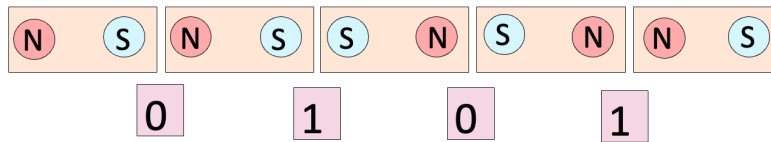


VS

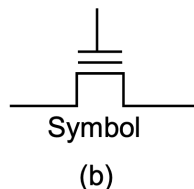
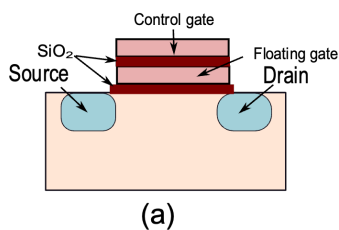


Dischi magnetici vs SSD

- Dischi magnetici: sequenza di piccoli magneti:



- SSD (Solid State Drive):



0: carica "intrappolata"

1: no carica "intrappolata"

SSD: aspetti generali

- Negli SSD *non* esistono parti in movimento (quindi non esistono ritardi di ricerca-rotazione);
- Distinzione tra **unità di I/O** (*pagina*, tipicamente 4KB) e **unità di cancellazione** (*blocco*, 64-256 unità di I/O);
- Ha prestazioni **asimmetriche**: le letture (*decine di us*) sono molto più veloci delle scritture (*centinaia di us*);
 - per scrivere una pagina, si deve prima cancellare un blocco: ciclo **program/erase (P/E)**;
 - una cella flash ha una durata di qualche centinaio di migliaia di cicli P/E.

Organizzazione di un SSD

- ❑ Il "core" è il componente FTL (gira su un processore con accesso a una memoria);
- ❑ L'FTL garantisce una **usura uniforme** e gestisce il **mapping** blocco logico - posizione fisica*;

